

CS 3006

Parallel and Distributed Computing

Lecture 4

Danyal Farhat
FAST School of Computing
NUCES Lahore

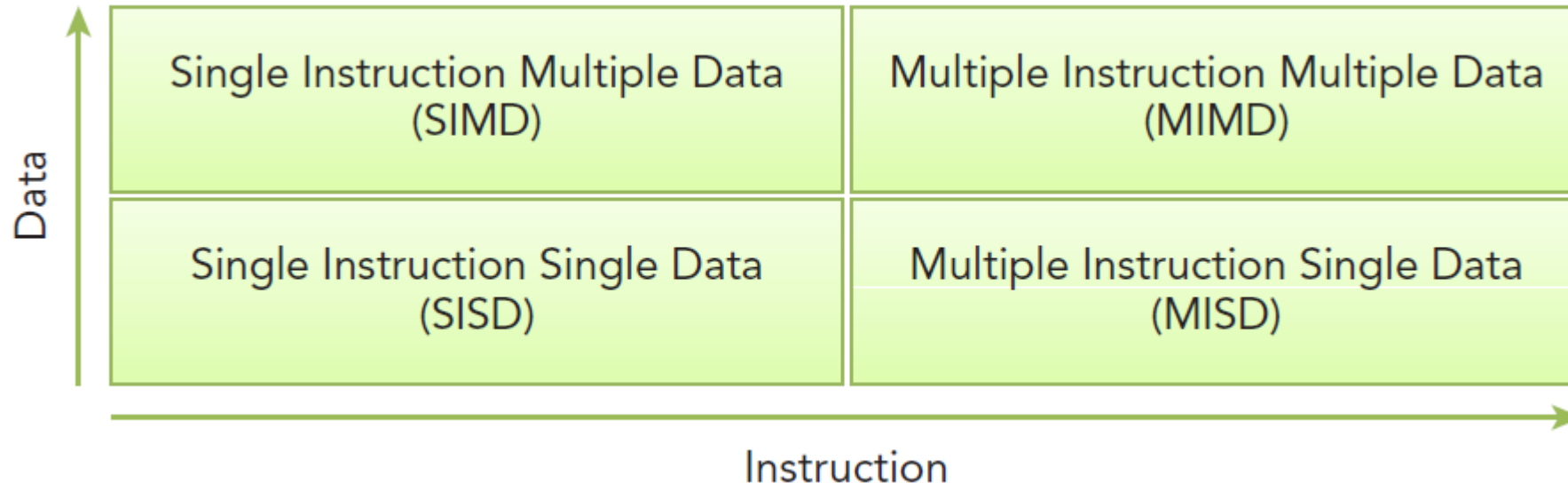
Flynn's Classical Taxonomy & Processor to Memory Connection Strategies

Lecture's Agenda

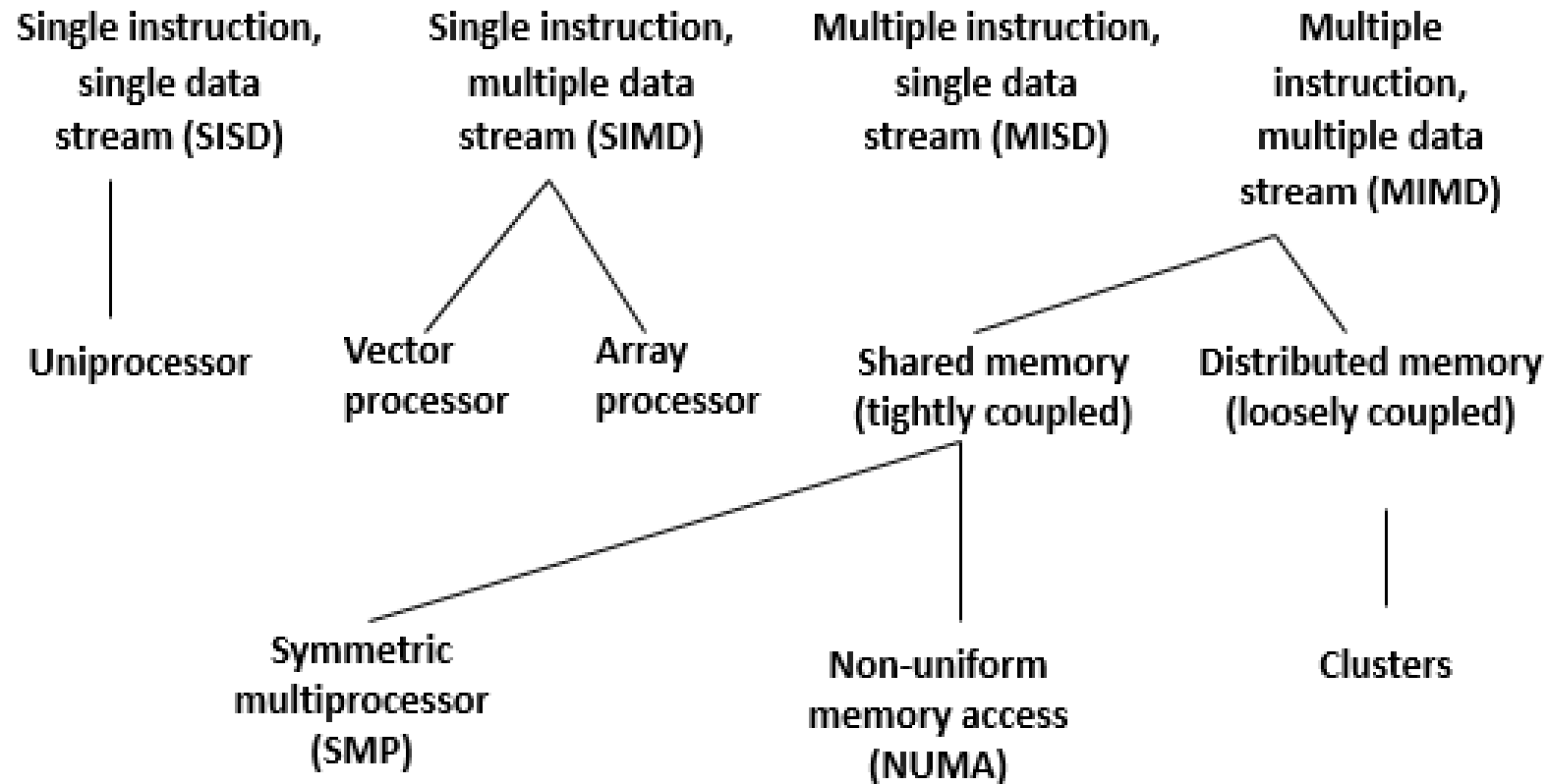
- **Flynn's Classical Taxonomy**
 - SISD
 - SIMD
 - MISD
 - MIMD
- **Physical Organization of Parallel Platforms**
 - PRAM
- **Routing Techniques and Costs**
- **Summary**
- **Additional Resources**

Flynn's Classical Taxonomy

- Widely used architectural **classification** scheme
- Classifies architectures into **four** types
- The classification is **based** on how data and instructions flow through the cores.
 - Instruction stream: Sequence of instructions from memory to control unit
 - Data stream: Sequence of data from memory to control unit



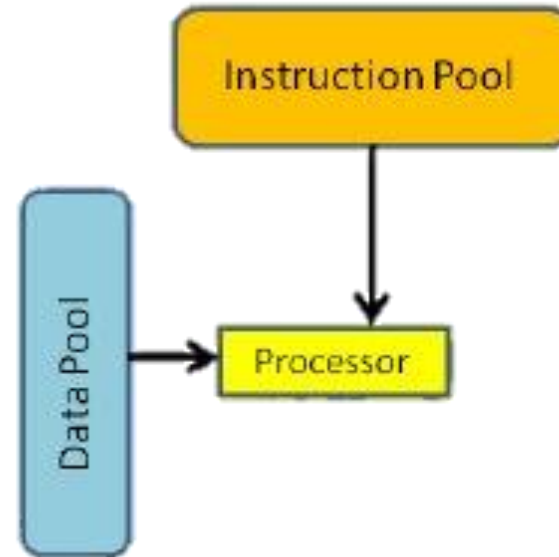
Processor Organizations



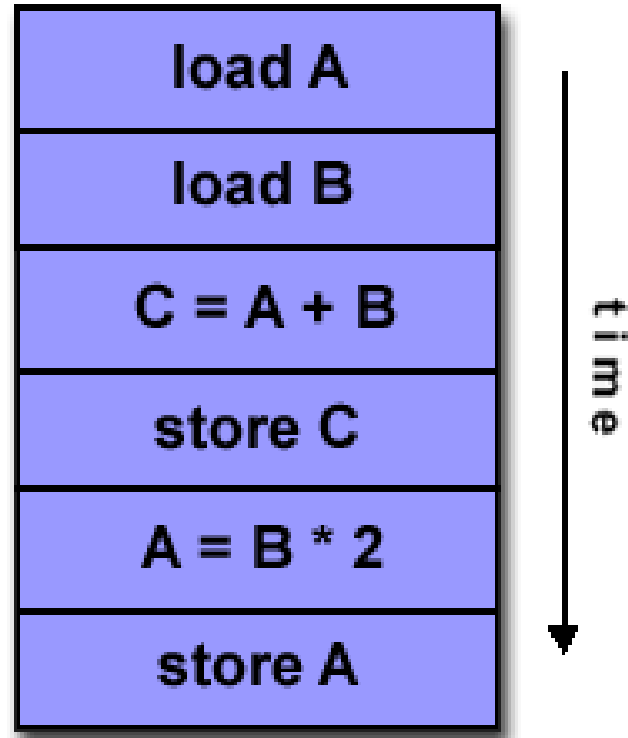
Flynn's Classical Taxonomy (Cont.)

SISD:

- Refers to **traditional** computer: a serial architecture
- This architecture includes **single core** computers
- Single instruction stream is in **execution** at a given time
- Similarly, only one data stream is **active** at any time



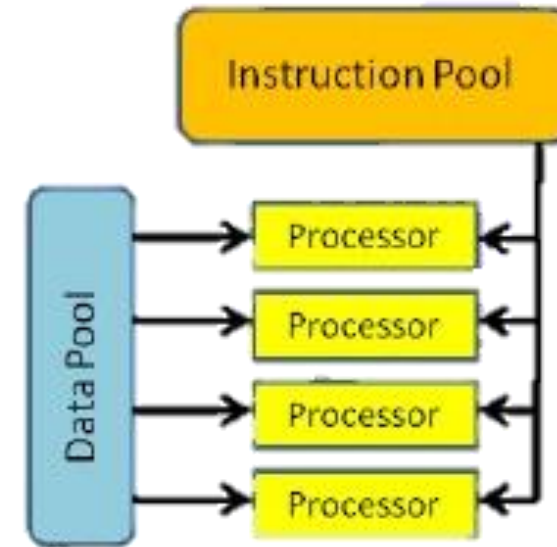
Example of SISD



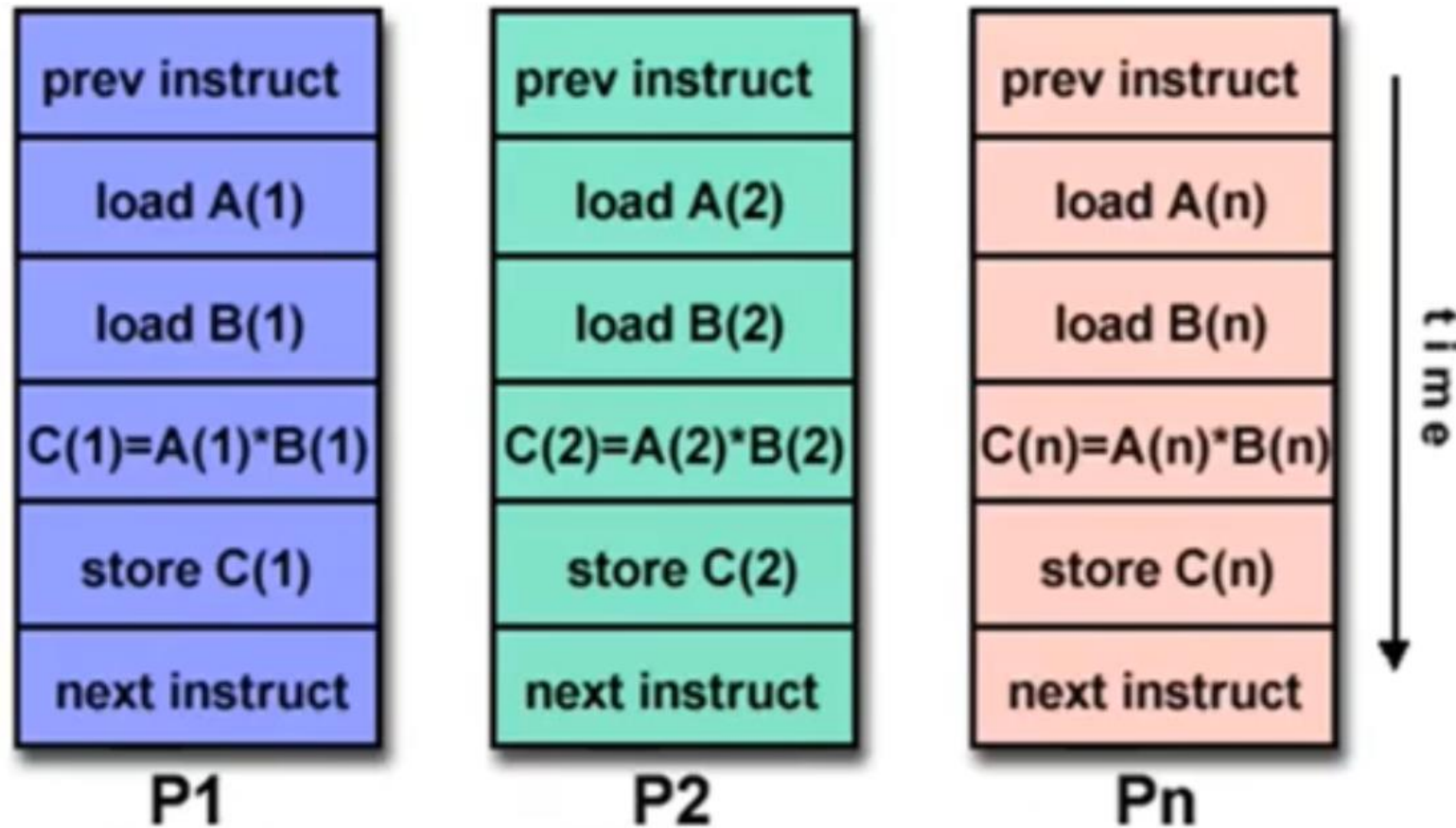
Flynn's Classical Taxonomy (Cont.)

SIMD:

- Refers to **parallel** architecture with multiple cores
- All the cores execute the **same instruction** stream at any time but, data stream is different for the each
- Well-suited for the **scientific operations** requiring large matrix-vector operations
- Vector computers (Cray vector processing machine) and Intel co-processing unit 'MMX' fall under this **category**
- Used with **array** operations, image processing and graphics



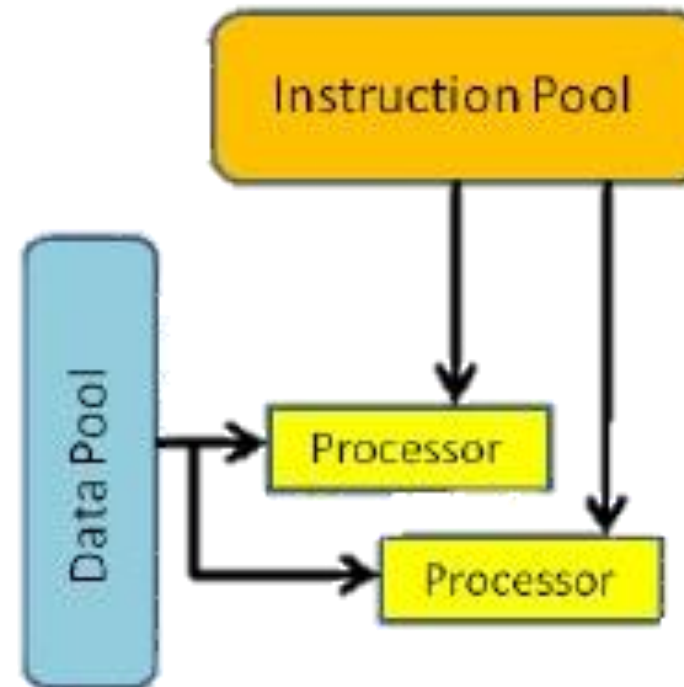
Example of SIMD



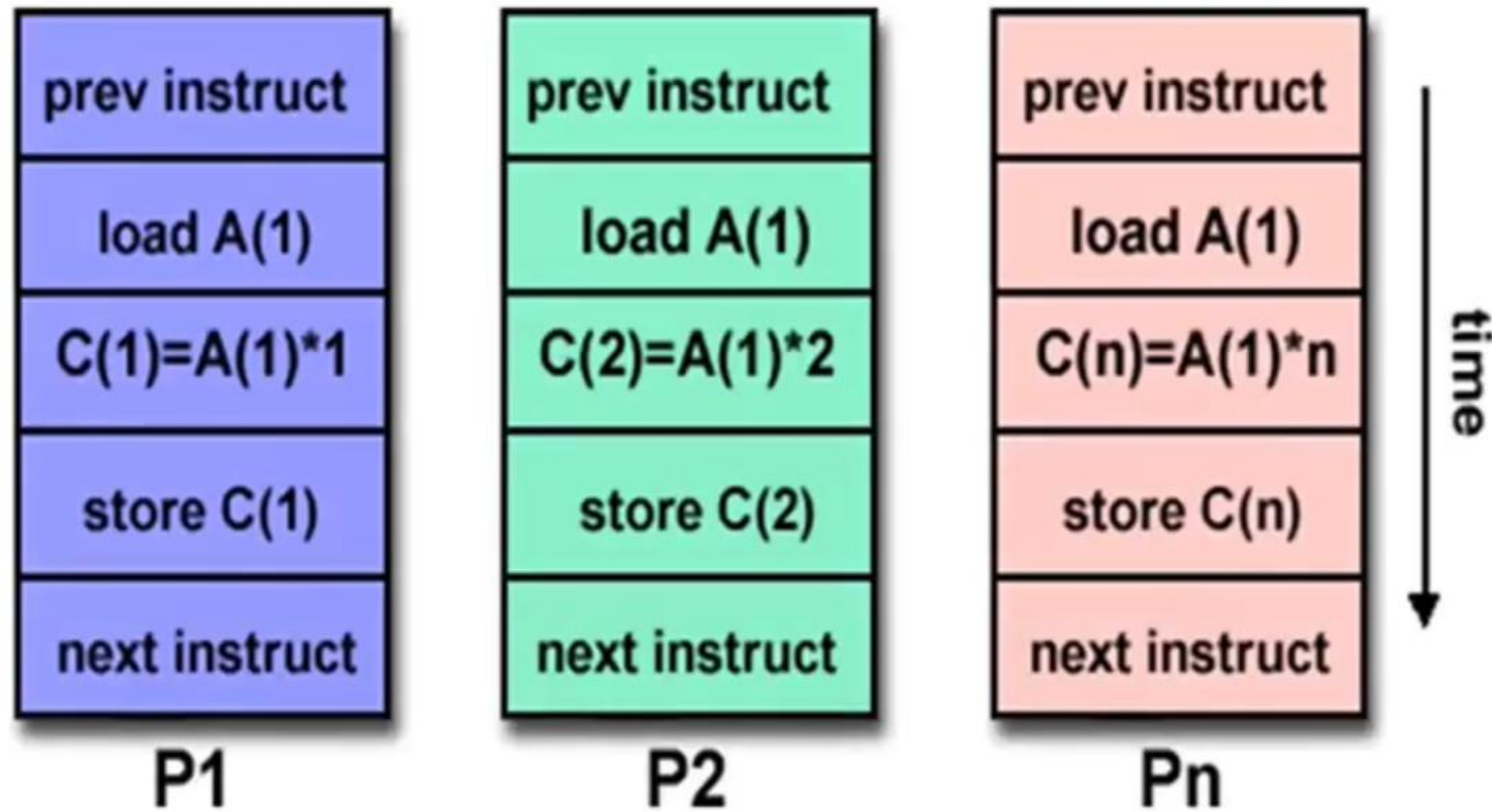
Flynn's Classical Taxonomy (Cont.)

MISD:

- Multiple instruction stream and single data stream
 - A pipeline of multiple independently executing **functional units**
 - Each operating on a single stream of **data** and forwarding results from one to the next
- Rarely used in practice
- **MISD Example:** Multiple algorithms attempting to crack a coded message



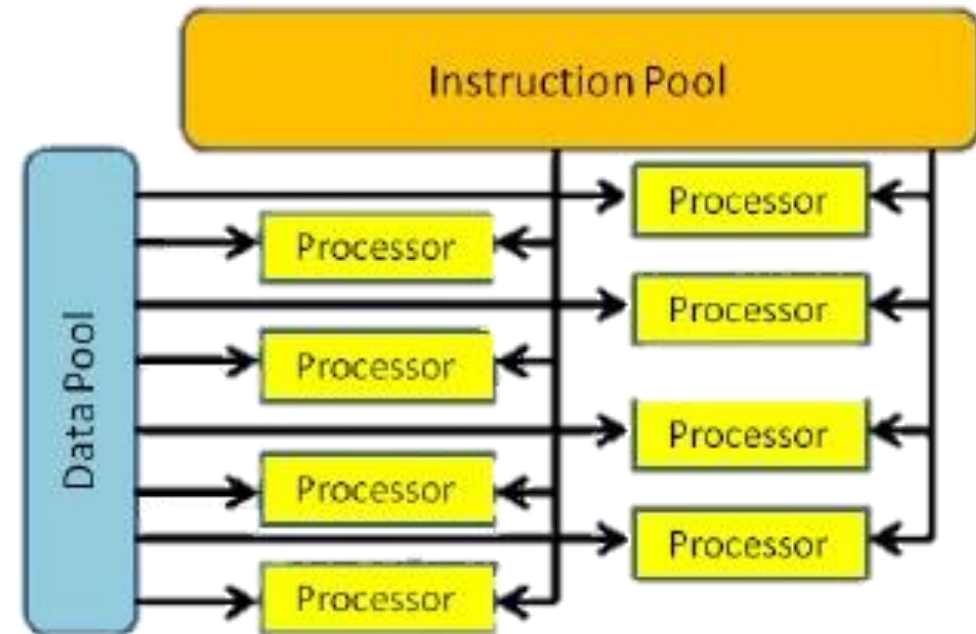
Example of MISD



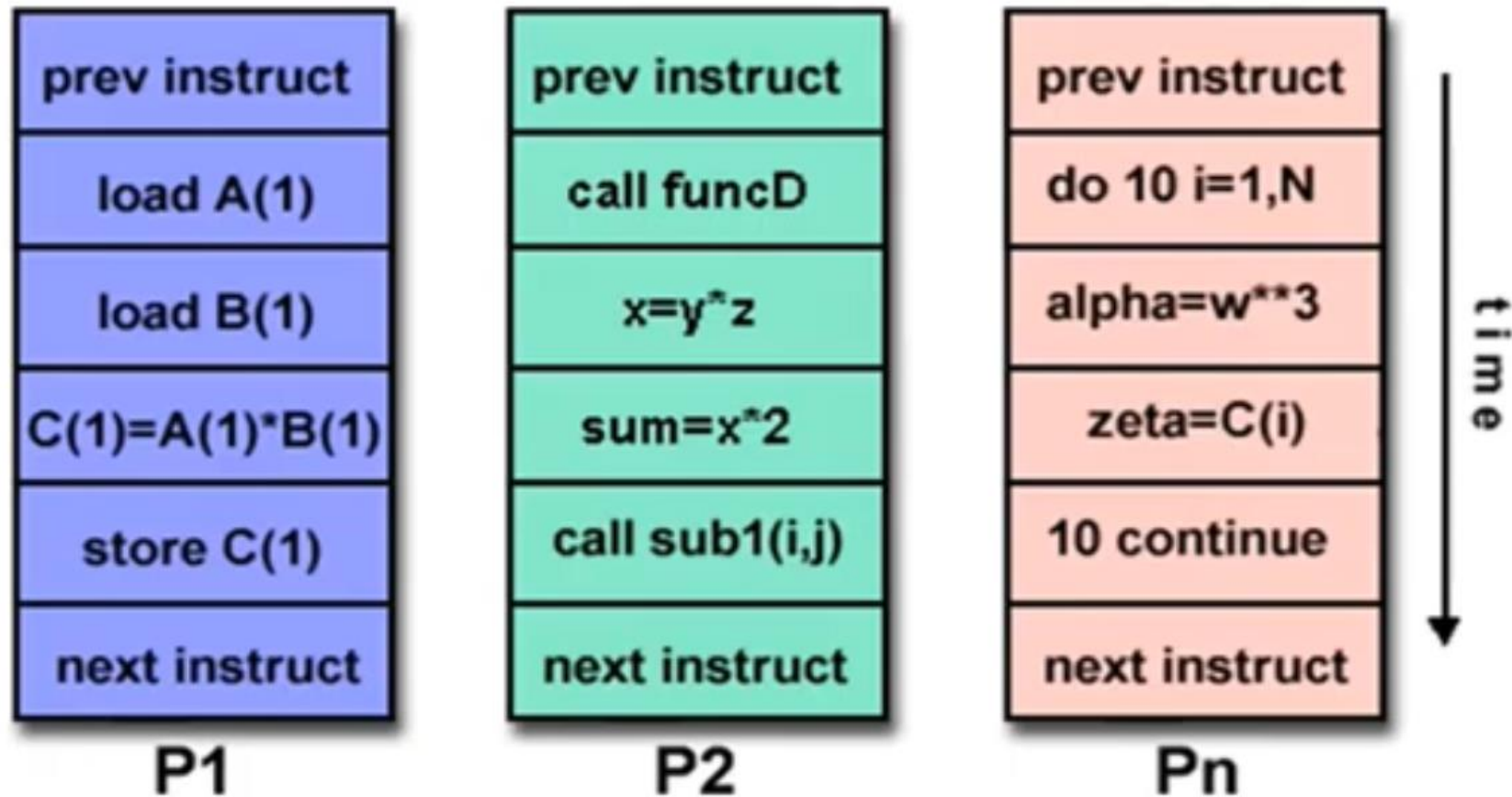
Flynn's Classical Taxonomy (Cont.)

MIMD:

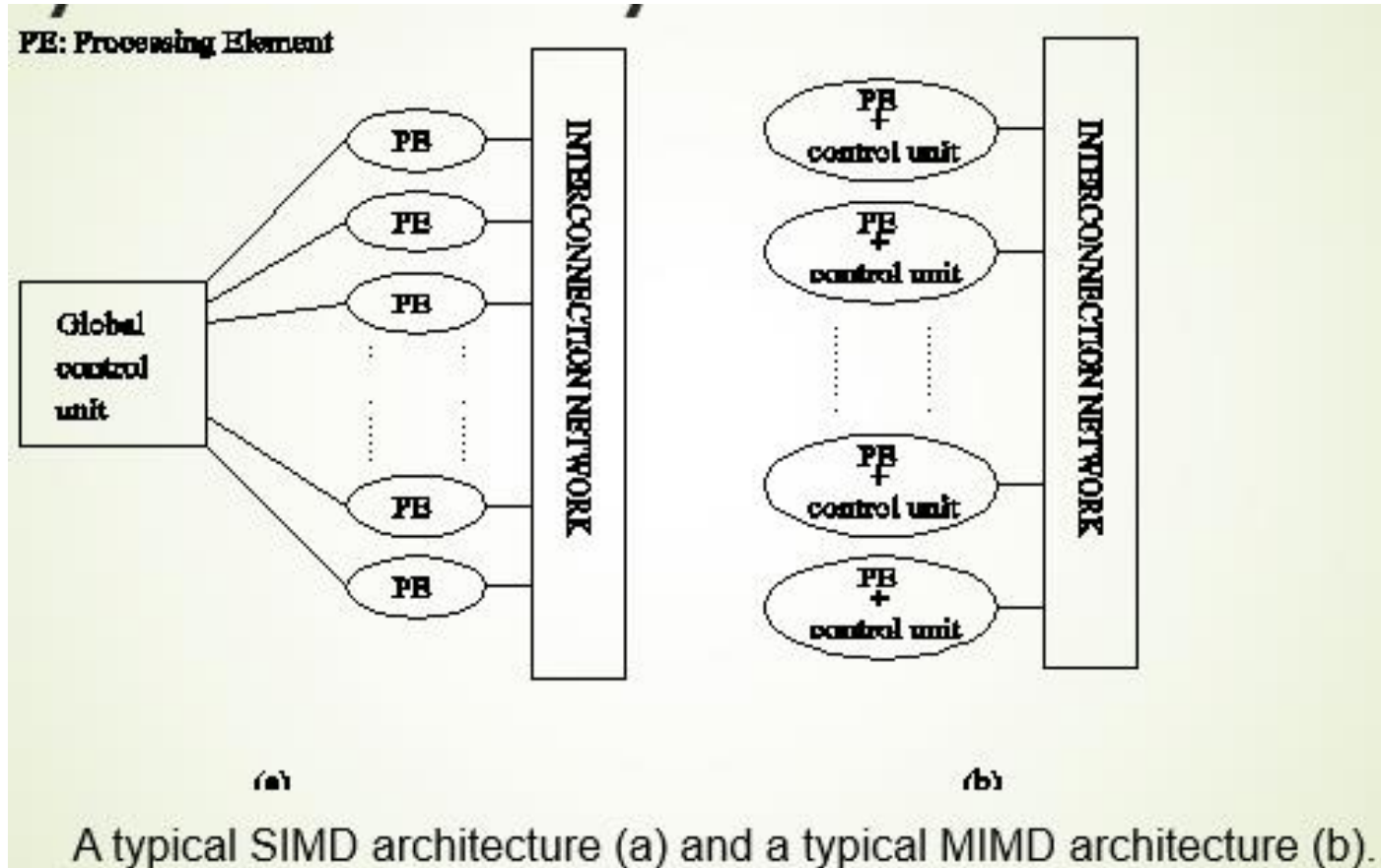
- Multiple **instruction** streams and multiple data streams
- Different CPUs can simultaneously **execute** different instruction streams manipulating different data
- Most of the modern parallel architectures fall under this category e.g., Multiprocessor and multicomputer architectures
- Many MIMD architectures **include** SIMD executions by default
- Supercomputers also fall in this category



Example of MIMD



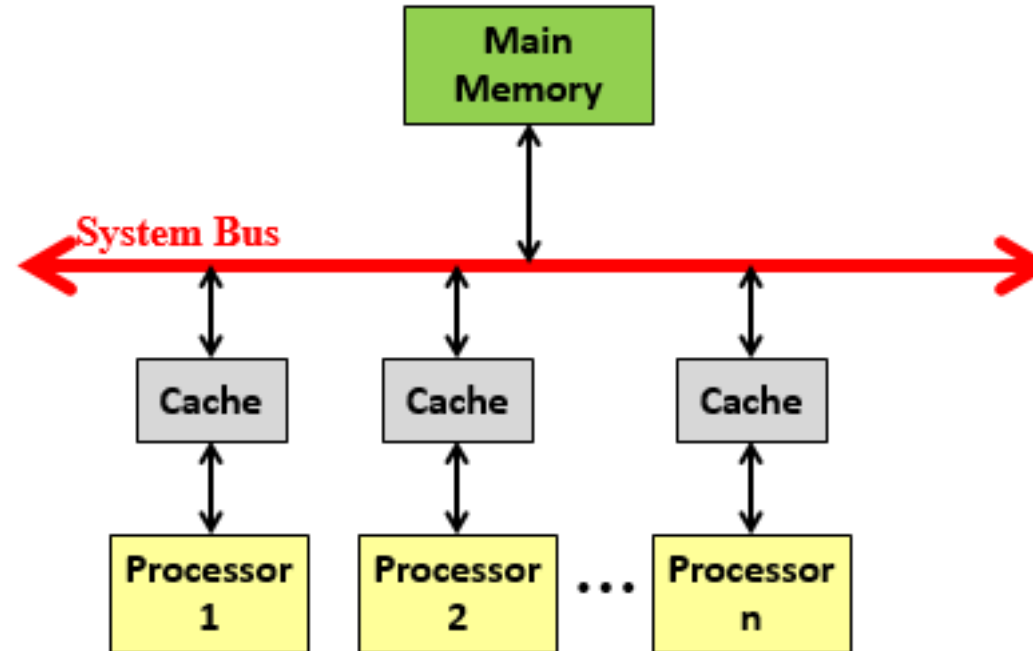
Flynn's Classical Taxonomy (Cont.)



SIMD-MIMD Comparison

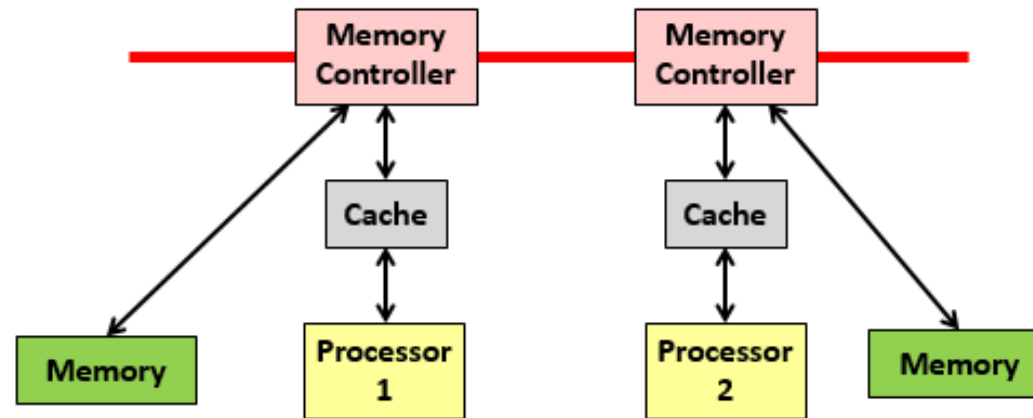
- SIMD computers require **less hardware** than MIMD computers (single control unit)
- However, since SIMD processors are specially designed, they tend to be **expensive** and have long design cycles
- Not all applications are naturally **suited** to SIMD processors
- In contrast, platforms supporting the SPMD (Same Program Multiple Data) paradigm can be **built** from inexpensive off-the-shelf components with relatively little effort in a short amount of time
 - The Term SPMD is close **variant** of MIMD

Uniform Memory Access (UMA)



- From all processing units to the shared memory, the **data access time** is constant
- Mostly **represented** by Symmetric Multiprocessor (SMP) machines

Non-Uniform Memory Access (NUMA)



- From all processing units to the shared memory, the **data access time** is **not constant**

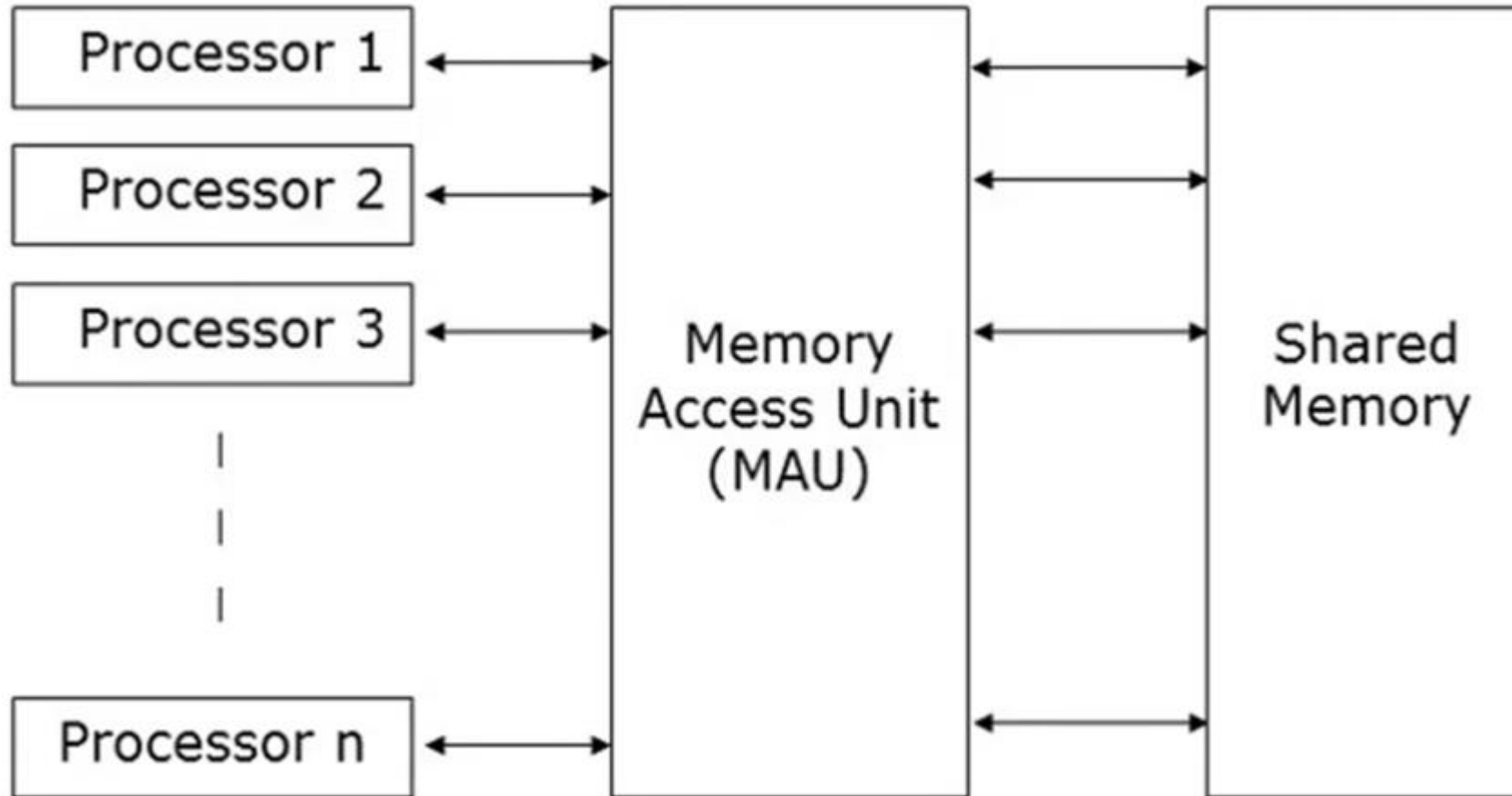
Physical Organization of Parallel Platforms

Architecture of an Ideal Parallel Computer

- **Parallel Random Access Machine (PRAM)**

- An **extension** to ideal sequential model: Random Access Machine (RAM)
- PRAMs **consist** of p processors
- A **global** memory
 - ✓ Unbounded **size**
 - ✓ Uniformly **accessible** to all processors with same address space
- Processors share a **common clock** but may execute different instructions in each cycle
- Based on simultaneous memory access mechanisms, PRAM can further be **classified**

Graphical Representation of PRAM



Parallel Random Access Machine (PRAM)

- PRAM has a set of **similar type** of processors
- Processors **communicate** with each other using the shared memory
- N processors can perform independent operations on N number of data in a given time, this might lead to **simultaneous access** of same memory location by different processors
 - To **solve** the simultaneous access of same memory location problem we have PRAM classes

PRAM Classes

- PRAMs can be divided into **four** classes

Exclusive-Read, Exclusive-Write (EREW) PRAM

- No two processors can perform **read/write** operations concurrently
- Weakest PRAM model, provides **minimum** memory access concurrency

Concurrent-Read, Exclusive-Write (CREW) PRAM

- All processors can read concurrently but **can't write** at same time
- Multiple **write accesses** to a memory location are serialized

Exclusive-Read, Concurrent-Write (ERCW) PRAM

- No two processors can perform read operations concurrently, but can **write**

Concurrent-Read, Concurrent-Write (CRCW) PRAM

- Most **powerful** PRAM model

PRAM Arbitration Protocols

- Concurrent reads do not create any semantic inconsistencies
- But, What about concurrent write?
- Need of an arbitration (mediation) mechanism to resolve concurrent write access

PRAM Arbitration Protocols (Cont.)

- **Common**

- Write only if all values that processors are attempting to write are **identical**

- **Arbitrary**

- Write the data from a **randomly selected** processor and ignore the rest

- **Priority**

- Follow a predetermined **priority order**
- Processor with **highest priority** succeeds and the rest fail

- **Sum**

- Write the **sum of the data items** in all the write requests
- The sum-based write conflict resolution model can be **extended** for any of the associative operators, that is defined for data being written

Physical Complexity of an Ideal Parallel Computer

- Processors and memories are **connected** via switches
- Since these switches must operate in $O(1)$ time at the level of words, for a system of p processors and m words, the switch **complexity** is $O(mp)$
 - Switches **determine** the memory word being accessed by each processor
 - Switch is a device that **opens or closes access** to certain data bank or word
- Clearly, for **meaningful** values of p and m , a true PRAM is not realizable

Communication Costs in Parallel Machines

- Along with idling (doing nothing) and contention (conflict e.g., resource allocation), communication is a major **overhead** in parallel programs
- The communication cost is usually **dependent** on a number of features including the following:
 - Programming **model** for communication
 - ✓ Required pattern of the communication in the program
 - Network **topology**
 - Data handling and **routing**
 - Associated network **protocols**
- Usually, distributed systems **suffer** from major communication overheads

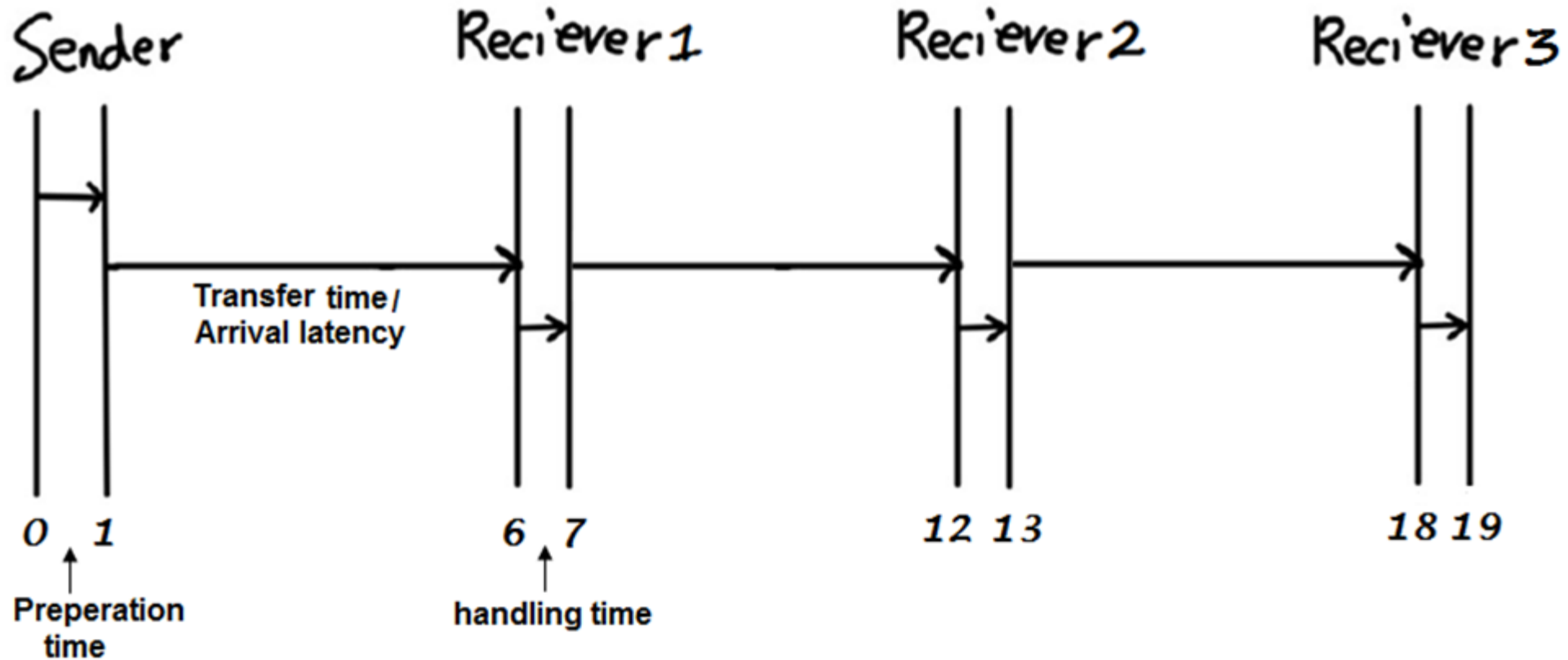
Message Passing Costs in Parallel Computers

- The **total time** to transfer a message over a network comprises of the following:
- **Startup time (t_s)**: Time spent at **sending and receiving nodes** (preparing the message [adding headers, trailers, and parity information], executing the routing algorithm, establishing interface between node and router, etc.)

Message Passing Costs in Parallel Computers (Cont)

- **Per-hop time (t_h):** This time is a function of number of **hops** (steps) and includes factors such as switch latencies, network delays, etc.
 - Also known as **node latency**
 - Also accounts for the latency to take **decision** of choosing next channel to which this message shall be forwarded
- **Per-word transfer time (t_w):** This time includes all overheads that are determined by the **length** of the message. This includes bandwidth of links, and buffering overheads, etc.
 - If channel bandwidth is r words/s then each word take $t_w = 1/r$ to traverse the link

Message Passing Costs in Parallel Computers (Cont)



Message Passing Costs in Parallel Computers (Cont)

Store-and-Forward Routing

- A message traversing multiple hops is **completely received** at intermediate hop before being forwarded to next hop
- The **total communication cost** for a message of size m words to traverse l communication links is

$$t_{comm} = t_s + (mt_w + t_h)l.$$

- In most platforms, t_h is small and the above expression can be approximated by

$$t_{comm} = t_s + mlt_w.$$

- *Cost of header transfer at each hop (step) t_h*
 - *mt_w is cost of transferring m words over the link*
- t_s is startup time*

Message Passing Costs in Parallel Computers (Cont)

Packet Routing

- Store-and-forward makes **poor use** of communication resources
- Packet routing **breaks messages** into packets and pipelines them through the network
- Since packets may take different paths, each **packet must carry** routing information, error checking, sequencing, and other related header information
 - Error checking (parity information), sequencing (order number)
 - Related headers: layers headers, addressing headers
- The **total communication time for packet routing** is approximated by:
$$t_{comm} = t_s + t_h l + t_w m.$$
- Here factor t_w also accounts for **overheads** in packet headers

Message Passing Costs in Parallel Computers (Cont)

Cut-Through Routing

- Takes the concept of packet routing to an extreme by further dividing messages into basic units called **flits** or flow control digits
- Since flits are typically small, the **header information** must be minimized
- This is done by forcing all flits to take the **same path**, in sequence
- A tracer message first **programs** all intermediate routers. All flits then take the same route

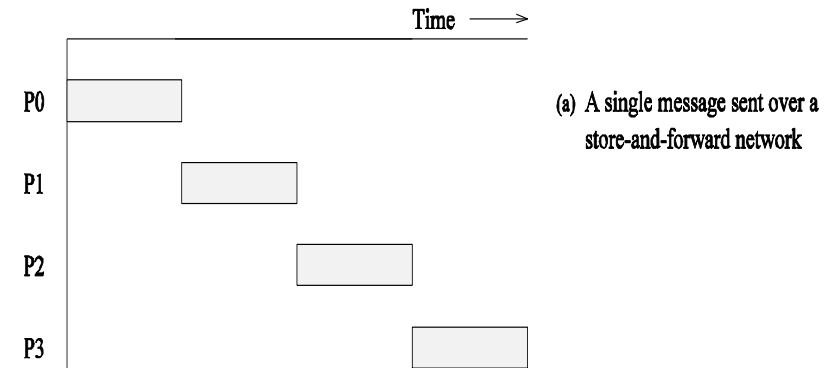
Message Passing Costs in Parallel Computers (Cont)

Cut-Through Routing (Cont.)

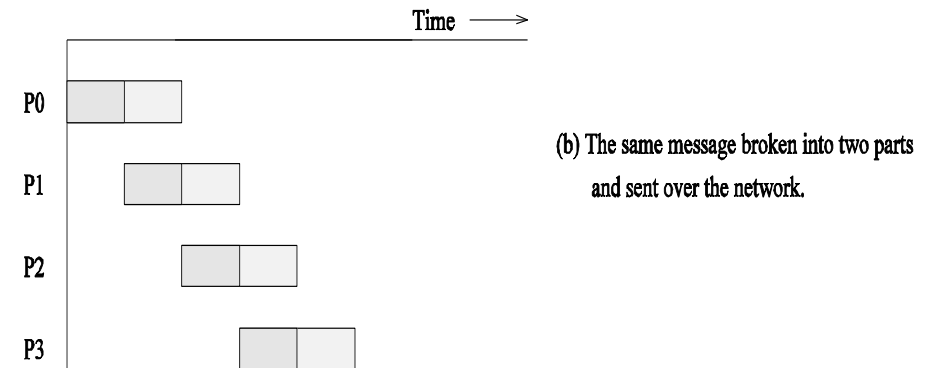
- Error checks are **performed** on entire message, as opposed to flits
- **No sequence numbers** are needed
 - Sequencing information is not needed as all the packets are following same path which ensures **in-order** delivery
- The **total communication time for cut-through routing** is approximated by:
$$t_{comm} = t_s + t_h l + t_w m.$$
- This is **identical** to packet routing, however, t_w is typically much smaller
 - Header of the message takes $l * t_h$ to reach the destination and entire message arrives in time $m t_w$ after the message header

Message Passing Costs in Parallel Computers (Cont.)

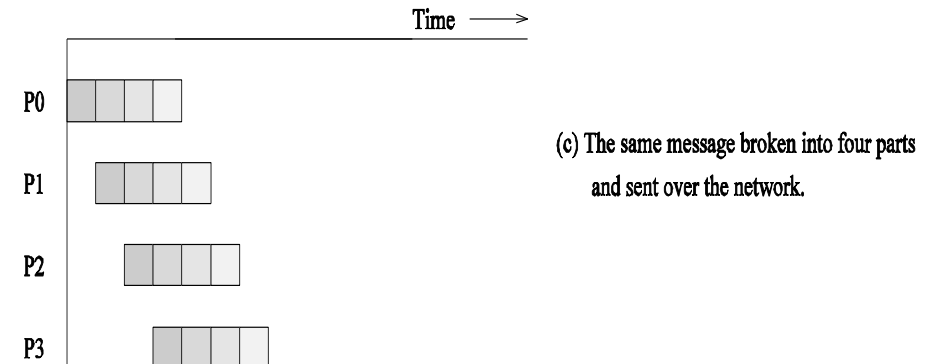
(a) through a store-and-forward communication network



b) and (c) extending the concept to cut-through routing



Shaded regions here **represent** the time where message is in transit (travel)
The **startup time** associated with this message transfer is assumed to be zero



Message Passing Costs in Parallel Computers (Cont)

Simplified Cost Model for Communicating Messages

- The **cost of communicating a message** between two nodes l hops away using cut-through routing is given by:

$$t_{comm} = t_s + lt_h + t_w m.$$

- In this expression, t_h is typically **smaller** than t_s and t_w . For this reason, the second term in the RHS does not show, particularly, when m is large
- For these reasons, we can approximate the **cost of message transfer** by:

$$t_{comm} = t_s + t_w m.$$

- For communication using flits, start-up time **dominates** the node latencies

Message Passing Costs in Parallel Computers (Cont)

Simplified Cost Model for Communicating Messages (Cont.)

- It is important to note that the original expression for communication time is **valid** for only uncongested networks
- Different communication patterns **congest** different networks to varying extents
- It is important to understand and **account** for this in the communication time accordingly

Summary

- **Flynn's Classical Taxonomy**

- Differentiates multiprocessor computers on the basis of dimensions of instruction and data

- **Processor Organizations**

- SISD - Used in uniprocessor computers
- SIMD - Used in vector or array processor computers
- MISD - Not commercially implemented
- MIMD - Used in supercomputers, grid computers etc.

Summary (Cont.)

- **SISD**

- Easy and deterministic but with limited performance
- SISD processor performance - MIPS Rate = $f \times \text{IPC}$
- How to increase performance of uniprocessor?

- **SIMD**

- Homogeneous processing units (vector or array processors)
- Execution of single instruction on multiple data sets using single control unit
- Associated data memory for each processing element

- **SIMD – Examples**

- Processing of pixels, online gaming servers, matrix based calculations etc.

Summary (Cont.)

- **MISD**

- Single data stream transmitted to a set of processors, each of which executes a different instruction sequence
- Commercially not implemented
- Example: Multiple cryptography algorithms attempting to crack a coded message

- **MIMD**

- Supercomputers, Grid computers, Networked parallel computers etc.

Summary (Cont.)

- **MIMD - Shared Memory - CPUs share same address space**
 - Uniform Memory Access (UMA) - Constant data access time
 - Non-Uniform Memory Access (NUMA) - Non-Constant data access time
- **MIMD - Distributed Memory**
 - CPUs connected via network and have their own associated memory
- **Architecture of an Ideal Parallel Computer – PRAM**
 - Extension to ideal sequential model: Random Access Machine (RAM)
 - Consist of p processors
 - Global memory (Unbounded size, Uniformly accessible to all processors with same address space)

Summary (Cont.)

- **PRAM Classes**

- Exclusive-Read, Exclusive-Write (EREW) PRAM
- Concurrent-Read, Exclusive-Write (CREW) PRAM
- Exclusive-Read, Concurrent-Write (ERCW) PRAM
- Concurrent-Read, Concurrent-Write (CRCW) PRAM

- **PRAM Arbitration Protocols**

- Concurrent writes do create semantic inconsistencies
- We need an arbitration mechanism to resolve concurrent write access
- Common, Arbitrary, Priority, Sum protocols can be used

Summary (Cont.)

- **Physical Complexity of an Ideal Parallel Computer**
 - Processors and memories are connected via switches
 - Switches must operate in $O(1)$ time at the level of words, for a system of p processors and m words, the switch complexity is $O(mp)$
 - For meaningful values of p and m , a true PRAM is not realizable
- **Communication Costs in Parallel Machines**

Summary (Cont.)

- **Message Passing Costs in Parallel Computers**

- Startup time, Per-hop time, Per-word transfer time

- **Store-and-Forward Routing** $t_{comm} = t_s + (mt_w + t_h)l.$

- **Packet Routing** $t_{comm} = t_s + t_hl + t_wm.$

- **Cut-Through Routing** $t_{comm} = t_s + t_hl + t_wm.$

- **Simplified Cost Model for Communicating Messages** $t_{comm} = t_s + t_wm.$

Additional Resources

- **Introduction to Parallel Computing** by Ananth Grama and Anshul Gupta

- **Chapter 2: Parallel Programming Platforms**

- ✓ **Section 2.3: Dichotomy of Parallel Computing Platforms**
- ✓ **Section 2.4: Physical Organization of Parallel Platforms**
- ✓ **Section 2.4.1: Architecture of an Ideal Parallel Computer**
- ✓ **Section 2.5 Communication Costs in Parallel Machines**
- ✓ **Section 2.5.1 Message Passing Costs in Parallel Computers**

Questions?